université
PARIS-SACLAY

RAPPORT DE TP

Instrumentation et interfaçage des systèmes embarqués

Auteur :

Gaëtan BOUCHY
Thomas ABOU JAOUDE

Enseignant :

M. EL OUARDI
M. GAC

Nous attestons que ce travail est original, qu'il est le fruit d'un travail commun en binôme et qu'il a été rédigé de manière autonome.

À Orsay, le 17/11/2022

université
PARIS-SACLAY

Sommaire

Sommaire 2
1 - Mise en œuvre d'une communication série asynchrone 3
1.1 Interfaçage aux périphériques et mise en œuvre des interruptions 3
1.1.1 Réalisation d'un chenillard avec temporisation bloquante 3
1.1.2 Réalisation d'un chenillard avec temporisation non-bloquante 4
1.1.3 Réalisation d'un chenillard avec touches commandant le sens de rotation 4
1.2 Mise en œuvre d'une communication série asynchrone 5
1.2.1 Code de l'automate émetteur 5
1.2.2 Code de l'automate récepteur 9
1.2.3 Croisement des fils Tx et Rx 13
1.2.4 Liaison série avec RTS et CTS 14
2 - Utilisation du protocole CAN 15
2.1 Réception de l'ID et du DLC d'une trame CAN 15
2.2 Affichage des IDs égaux à 0x310 15
2.3 Affichages des IDs égaux à 0x320 et 0x330 16
2.4 Affichage de deux octets de données sur le barre-graph (ID = 0x370) 16
2.5 Décodage des trames (sauf ID = 0x370) 16
2.6 Émission de trames CAN en appuyant sur une touche 17
2.7 Envoie d'un message de taille allant jusqu'à 30 octets 17
3 - TP d'interface PCI-9112 19
3.1 Acquisition d'un signal issu d'un GBF et affichage des échantillons en temps réel 19
3.2 Écart de tension entre les deux signaux 20
3.3 Explication du nombre maximum d'échantillons par seconde 21
3.4 Génération logicielle d'un signal de forme triangulaire et affichage des échantillons en temps réel 22
3.5. Code mesurant la fréquence, l'amplitude et la valeur moyenne d'un signal 23
4 - Annexes 26
4.1 Code pour la mise en œuvre d'une communication série asynchrone 26
4.2 Code pour l'utilisation du protocole CAN 30
4.3 Code d'interface PCI-9112 37

1 - Mise en œuvre d'une communication série asynchrone

1.1 Interfaçage aux périphériques et mise en œuvre des interruptions

1.1.1 Réalisation d'un chenillard avec temporisation bloquante

La réalisation d'un chenillard avec temporisation bloquante s'est déroulée en trois étapes :
- L'initialisation des variables (LEDs, tableau de int pour l'affichage et les deux boucles for)
- La fonction de temporisation bloquante
- Le main qui utilise les interruptions sur le Timer0, affiche sur les motifs sur le barre-graph et appelle la fonction de temporisation bloquante

```
xdata int leds _at_ 0xFD00;
code int tab[10] = {0x000,0x001,0x003,0x00F,0x000,0x001,0x003,0x00F,0x000,0x001};
int i = 0, j = 0;
```

Figure 1 : Initialisation des variables

```
void tempoBloquante()
{
    for(i=0; i<30000; i++){}
}
```

Figure 2 : Fonction de temporisation bloquante

```
void main (void)
{
init_timer0();

while (1)
{
    if(FLAG_Timer==1)
    {
        leds = tab[j];
        j++;
        FLAG_Timer=0;
    }

    if(j==10)
        j=0;

    tempoBloquante();
}
}
```

Figure 3 : programme « Main »

Dans notre programme main, nous initialisons les interruptions sur le Timer0. Lorsque ce dernier atteint une certaine valeur, il déclenche un « flag » et va mettre la variable FLAG_Timer à 1. À ce moment-là, nous affichons les motifs sur le barre-graph puis incrémentons j pour en afficher un nouveau lors de la prochaine itération. Entre chaque itération, nous appelons la fonction tempoBloquante() qui consiste en une boucle for de 30 000 et permet de ralentir le programme. C'est une fonction de temporisation bloquante car une fois que nous sommes rentrés dans le boucle, il est impossible d'en sortir avant la fin des 30 000 itérations.

1.1.2 Réalisation d'un chenillard avec temporisation non-bloquante

Contrairement au programme précédent, la seule modification consiste en la réalisation d'une temporisation non-bloquante. Pour cela, nous incrémentons un compteur qui est retourné à chaque itération (« return 0 »). Lorsque ce dernier atteint la valeur de 5 000, nous retournons alors la valeur 1 (« return 1 »). Cela sera notre condition pour l'affichage des motifs sur notre barre-graph (« flag==1 »).

```
int tempoNonBloquante()
{
    compteur++;
    if(compteur==5000)
    {
        compteur=0;
        return 1;
    }
    else
        return 0;
}
```

Figure 4 : Fonction de temporisation non-bloquante

```
void main (void)
{  /* debut du programme apres le startup  */
init_timer0();

while (1)
{
    if(FLAG_Timer==1&&flag==1)
    {
        j++;
        leds = tab[j];
        FLAG_Timer=0;
    }

    if(j==10)
        j=0;

    flag = tempoNonBloquante();
}
}
```

Figure 5 : Programme main

1.1.3 Réalisation d'un chenillard avec touches commandant le sens de rotation

Pour réaliser un chenillard avec touches commandant le sens de rotation, nous aurons besoin des fonctions detect_touche() qui détectent l'appuie sur une touche et decode_touche() qui nous indiquent de quelles touches il s'agit.

```
void chenillard()
{
    if(decode_touche(detect_touche)==9)
        sens = 0;

    if(decode_touche(detect_touche)==8)
        sens = 1;
}
```

Figure 6 : Fonction chenillard qui va définir le sens de rotation en fonction de la touche appuyée

```
void main (void)
{  /* debut du programme apres le startup  */
init_timer0();

while (1)
{
    chenillard(); /// Appelle de la fonction chenillard

    if(FLAG_Timer==1&&flag==1)
    {   /// Incrémentation ou décrémentation en fonction du sens
        if(sens==0)
            j++;
        else
            j--;

        leds = tab[j];
        FLAG_Timer=0;
    }

    if(j==10&&sens==0) /// RAZ en fonction du sens
        j=0;

    if(j==0&&sens==1) /// mise à 10 de j en fonction du sens
        j=10;

    flag = tempoNonBloquante();
}
}
```

Figure 7 : Programme main

Pour expliquer les programmes, lorsque nous appuyons sur la touche 9 de la carte, cela met la variable « sens » à 0. Cette variable va par la suite gérer l'incrémentation de la variable j qui gère l'affichage des motifs sur le barre-graph. Si nous appuyons sur la touche 8 de la carte, cela met la variable « sens » à 1. Cette variable va par la suite gérer la décrémentation de la variable j changeant ainsi le sens de rotation.

Concernant les interruptions, le temps maximum entre deux interruptions matérielles du Timer0 est de 192 µs. Ainsi si nous voulons un flag logiciel d'une période de 250 ms, nous devons calculer le nombre d'itérations d'interruptions logicielles matérielles nécessaires soit : $\boldsymbol{Nombre\ d'itérations} = \frac{250\ ms}{192\ \mu s} = \frac{250\,000\ \mu s}{192\ \mu s} = \mathbf{1302}$, c'est pourquoi nous retrouvons cette valeur de 1302 dans le programme d'interruption de Timer0.

```
void timer0() interrupt 1
{
    i=i+1;
    if(i>=1302)
    {
        i=0;
        FLAG_Timer=1;
    }
}
```

Figure 8 : Programme d'interruption de Timer0

1.2 Mise en œuvre d'une communication série asynchrone

1.2.1 Code de l'automate émetteur

Pour réaliser le code de l'automate émetteur, nous regardons tout d'abord de quoi se compose une trame d'émission :

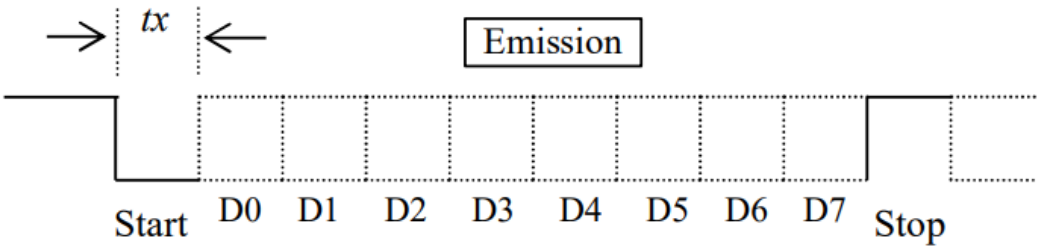


Figure 9 : chronogramme du signal vu en émission

Ainsi, comme nous pouvons le voir, une trame d'émission se compose d'un bit de start, des 8 bits de la donnée et terminée par un bit de stop. Nous devrons alors retrouver cela sur notre oscilloscope.

À partir de l'automate émetteur fourni, nous pouvons réaliser notre code qui sera composé d'un switch avec les différents cas :

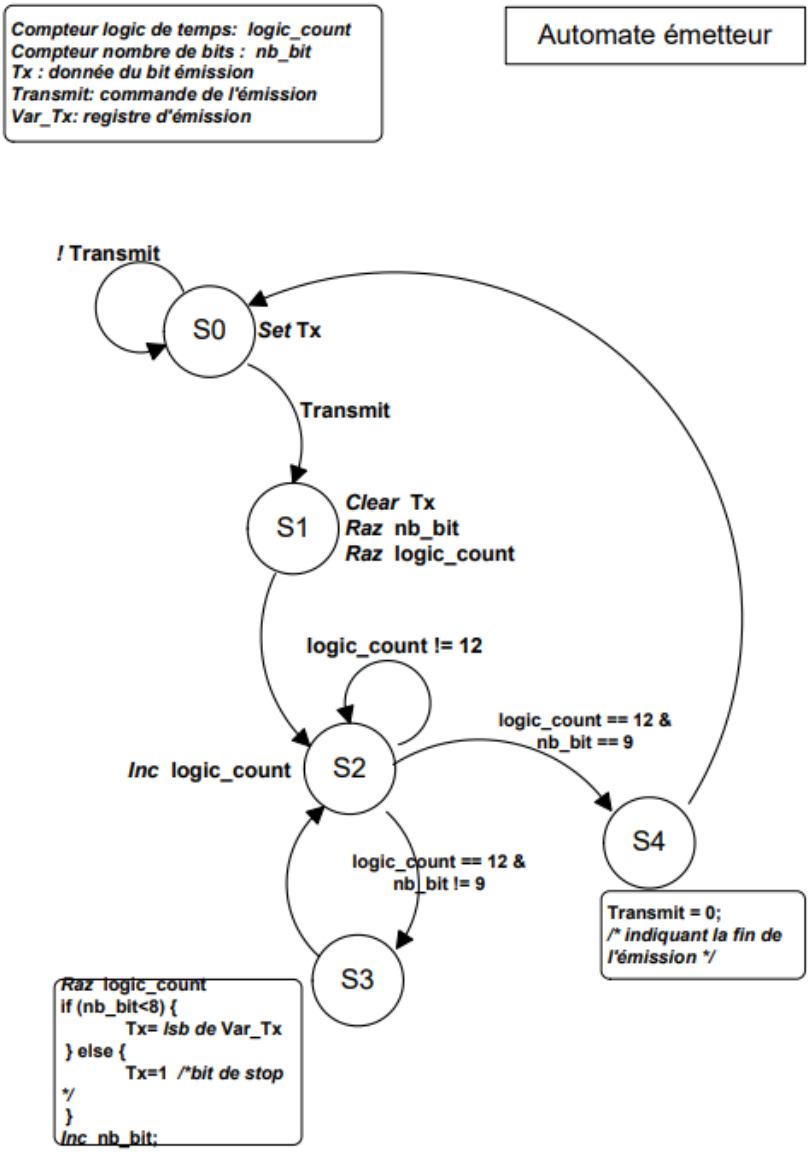


Figure 10 : Schéma de l'automate émetteur

Nous allons maintenant détailler notre switch. Dans le cas 0, nous mettons Tx à 1 et nous restons dans ce cas tant que « transmit » vaut 0. Si « transmit » vaut 1, nous passons alors au cas 1. Cela se traduit ainsi en code :

```
void automate_emission(){

    switch(emet){
        case 0:
            Tx = 1;
            if(transmit==1)
                emet = 1;
            break;
```

Figure 11 : Automate émetteur, cas 0

Pour le cas 1, nous devons clear « Tx » donc le mettre à 0 puis faire une remise à zéro de « nb_nit » et « logic_count » puis nous passons au cas 2. Cela se traduit ainsi en code :

```
case 1:
    Tx=0;
    nb_bit = 0;
    logic_count = 0;
    emet =2;
    break;
```

Figure 12 : Code de l'automate émetteur, cas 1

Pour le cas 2, nous incrémentons « logic_count » et tant que celui-ci ne vaut pas 12, nous restons dans le cas 2. Si « logic_count » vaut 12 et « nb_bit » ne vaut pas 9, alors nous allons dans le cas 3. Si « logic_count » vaut 12 et « nb_bit » vaut 9, nous allons dans le cas 4. Cela se traduit ainsi en code :

```
case 2:
    if(logic_count==12)
    {
        if(nb_bit!=9)
            emet = 3;
        else
            emet = 4;
     }

    logic_count++;
    break;
```

Figure 13 : Code de l'automate émetteur, cas 2

Pour le cas 3, nous faisons une remise à zéro de « logic_count ». Si « nb_bit » est strictement inférieur à 8, « Tx » prends la valeur du bit de poids faible de « var_tx » puis nous faisons un décalage vers la droite de « var_tx » tel que (11001100 devient alors X1100110). Si « nb_bit » est supérieur ou égal à 8, alors « Tx » prends la valeur 1 (valeur du bit de stop). Puis nous incrémentons « nb_bit » puis nous retournons dans le cas 2. Cela se traduit ainsi en code :

```
case 3:
    logic_count=0;
    if (nb_bit<8)
    {
        Tx = var_tx & 0x01;
        var_tx = var_tx >> 1;
    }
    else
        Tx=1;

    nb_bit++;
    emet=2;
    break;
```

Figure 14 : Code de l'automate émetteur, cas 3

Pour le cas 4, nous mettons « transmit » à 0 et nous retournons au cas 0. Cela se traduit ainsi en code :

```
case 4:
    transmit=0;
    emet = 0;
    break;
```

Figure 15 : Code de l'automate émetteur, cas 4

Si nous mettons bout à bout les quatre cas que nous avons vu précédemment, nous obtenons le code final de l'automate pour l'émission :

```
void automate_emission(){

    switch(emet){
        case 0:
            Tx = 1;
            if(transmit==1)
                emet = 1;
            break;
        case 1:
            Tx=0;
            nb_bit = 0;
            logic_count = 0;
            emet =2;
            break;
        case 2:
            if(logic_count==12)
            {
                if(nb_bit!=9)
                    emet = 3;
                else
                    emet = 4;
             }

            logic_count++;
            break;
       case 3:
           logic_count=0;
           if (nb_bit<8) {
               Tx= var_tx & 0x01;
               var_tx = var_tx>>1;
           }
           else
               Tx=1;

           nb_bit++;
           emet=2;
           break;
       case 4:
           transmit=0;
           emet = 0;
           break;
    }
}
```

Figure 16 : Code de l'automate émetteur

Il ne nous reste plus qu'à faire le main qui vérifie si l'on appuie sur une touche puis qui va l'afficher via « var_tx ». Nous remettons ensuite la variable « transmit » à 1 pour indiquer que le programme ne peut pas envoyer de caractère. La fonction automate_emission() est appelée automatiquement via interruption du Timer0 :

```
Toucheclavier = detect_touche();
if(Toucheclavier!=(char)0xff)
{
    var_tx = decode_touche(Toucheclavier);
    printf("touche=%u\n",(unsigned int)var_tx);
    transmit = 1;
}
```

Figure 17 : Programme main de l'automate émetteur

1.2.2 Code de l'automate récepteur

Pour réaliser le code de l'automate récepteur, nous regardons tout d'abord de quoi se compose une trame de réception :

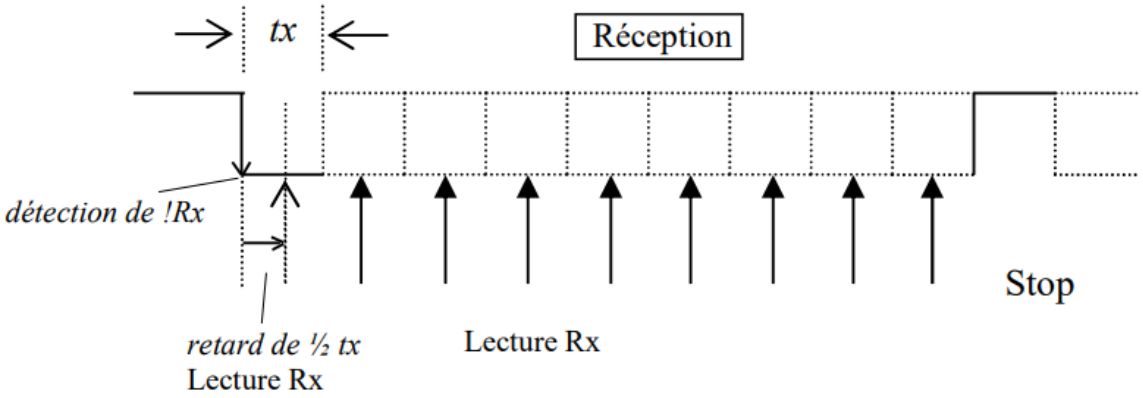


Figure 18 : chronogramme du signal vu en réception

Ainsi, comme nous pouvons le voir, une trame de réception se compose d'un bit de start (détecté de manière synchrone), des 8 bits de la donnée et terminée par un bit de stop. Nous devrons alors retrouver cela sur notre oscilloscope.

À partir de l'automate récepteur fourni, nous pouvons réaliser notre code qui sera composé d'un switch avec les différents cas :

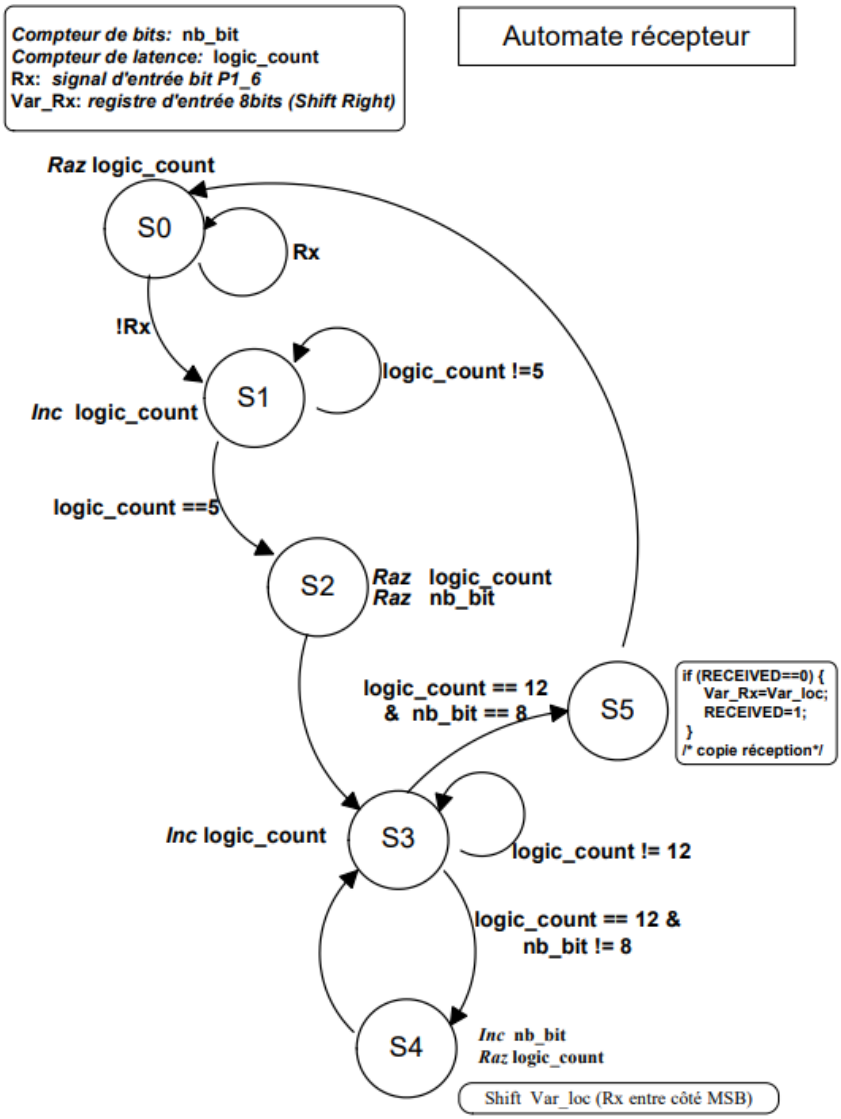


Figure 19 : Schéma de l'automate récepteur

Nous allons maintenant détailler notre switch. Dans le cas 0, nous faisons une remise à zéro de « logic_count ». Puis si « Rx » vaut 0, nous passons au cas 1, sinon, nous restons dans le cas 0. Cela se traduit ainsi en code :

```
switch(recept){
    case 0:
        logic_countR =0;
        if(Rx==0)
            recept=1;
        break;
```

Figure 20 : Code de l'automate récepteur, cas 0

Pour le cas 1, nous devons incrémenter « logic_count », tant que celui-ci n'est pas égal à 5, nous restons dans le cas 1, si celui-ci vaut 5, nous allons au cas 2. Cela se traduit ainsi en code :

```
case 1:
    if(logic_countR==5)
        recept=2;

    logic_countR++;
    break;
```

Figure 21 : Code de l'automate récepteur, cas 1

Pour le cas 2, nous devons faire une remise à zéro de « logic_count » et de « nb_bit » puis nous allons au cas 3. Cela se traduit ainsi en code :

```
case 2:
    logic_countR = 0;
    nb_bitR = 0;
    recept=3;
    break;
```

Figure 22 : Code de l'automate récepteur, cas 2

Pour le cas 3, nous incrémentons « logic_count » et tant que celui-ci ne vaut pas 12, nous restons dans le cas 3. Si « logic_count » vaut 12 et « nb_bit » ne vaut pas 8, alors nous allons dans le cas 4. Si « logic_count » vaut 12 et « nb_bit » vaut 8, nous allons dans le cas 5. Cela se traduit ainsi en code :

```
case 3:
    if( logic_countR==12)
    {
        if(nb_bitR!=8)
        {   recept = 4;

        }
        else
            recept = 5;
    }

    logic_countR++;
    break;
```

Figure 23 : Code de l'automate récepteur, cas 3

Pour le cas 4, nous incrémentons « nb_bit » et faisons une remise à zéro de « logic_count ». Puis nous faisons un décalage de « Var_loc ». Si « rx » vaut 0, alors « Var_loc » prends la valeur du bit de poids fort de « Var_loc » puis nous revenons au cas 3. Cela se traduit ainsi en code :

```
case 4:
    nb_bitR++;
    logic_countR = 0;
    Var_loc = Var_loc >> 1;

    if(Rx==1)
        Var_loc = Var_loc | 0x80;

    recept = 3;
    break;
```

Figure 24 : Code de l'automate récepteur, cas 4

Pour le cas 5, si « received » est égal à 0, alors « var_rx » prend la valeur de « Var_loc » puis « received » prends la valeur 1. Puis nous allons au cas 0. Cela se traduit ainsi en code :

```
case 5:
    if(RECEIVED==0)
    {
        var_rx=Var_loc;
        RECEIVED=1;
    }
    recept = 0;
    break;
```

Figure 25 : Code de l'automate récepteur, cas 5

Si nous mettons bout à bout les cinq cas que nous avons vus précédemment, nous obtenons le code final de l'automate pour la réception :

```
void automate_reception()
{
short Var_loc;
    switch(recept){
        case 0:
            logic_countR =0;
            if(Rx==0)
                recept=1;
            break;
        case 1:
            if(logic_countR==5)
                recept=2;

            logic_countR++;
            break;
        case 2:
            logic_countR = 0;
            nb_bitR = 0;
            recept=3;
            break;
        case 3:
            if( logic_countR==12)
            {
                if(nb_bitR!=8)
                    recept = 4;
                else
                   recept = 5;
            }

            logic_countR++;
            break;
       case 4:
            nb_bitR++;
            logic_countR = 0;
            Var_loc = Var_loc >> 1;

            if(Rx==1)
                Var_loc = Var_loc | 0x80;

            recept = 3;
            break;
       case 5:
            if(RECEIVED==0)
            {
                var_rx=Var_loc;
                RECEIVED=1;
            }
            recept = 0;
            break;
    }
}
```

Figure 26 : **Code de l'automate récepteur**

Il ne nous reste plus qu'à faire le main qui va afficher la touche reçue via la variable « var_rx ». Nous remettons ensuite la variable « received » à 0 pour indiquer à l'automate récepteur qu'un autre caractère peut être transféré dans la variable d'échange. La fonction automate_réception() est appelée automatiquement via interruption du Timer0 :

```
if(RECEIVED==1)
{
    messageRecu = var_rx;
    printf("message recu=%u\n",(unsigned int)messageRecu);
    RECEIVED=0;
}
```

Figure 27 : Programme main de l'automate récepteur

1.2.3 Croisement des fils Tx et Rx

En strappant les pins Rx et Tx de notre carte, cela signifie qu'un message envoyé via Tx peut-être reçu via Rx.

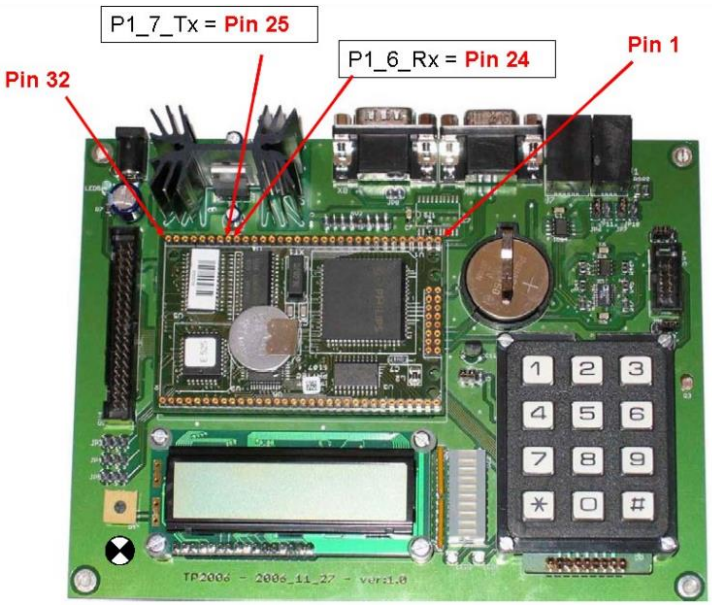


Figure 28 : Schéma de la carte

Dans notre code, nous affichons tout d'abord la touche transmise puis nous affichons par la suite le message reçu.

```
toucheClavier = detect_touche();
if(toucheClavier!=(char)0xff)
{
    var_tx =decode_touche(toucheClavier);
    printf("touche=%u\n",(unsigned int)var_tx);
    transmit = 1;
}

if(RECEIVED==1)
{
    messageRecu = var_rx;
    printf("message recu=%u\n",(unsigned int)messageRecu);
    RECEIVED=0;
}
```

Figure 29 : Code main pour la transmission et la réception d'un message

Si l'on essaie d'envoyer le message « 1 » en appuyant sur la touche donc le code binaire « 00000001 ». Cela se déroule ainsi au niveau de la console :

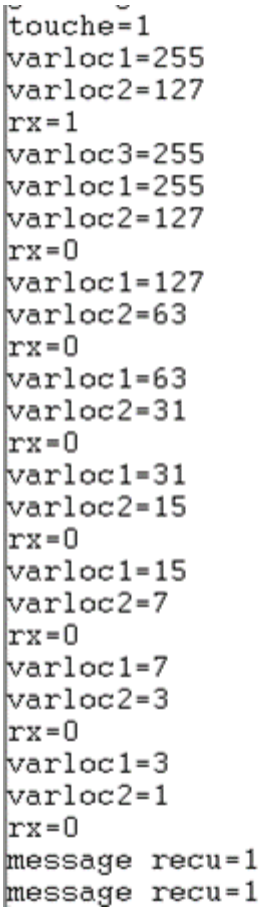


Figure 30 : Console montrant un message envoyé et un message reçu

On voit que la touche appuyée donc envoyée est 1. Puis l'on commence à transmettre le message avec varloc1 et varloc2 qui montrent l'avancement du message. On peut donc voir que le « 00000001 » en binaire rentre côté MSB et que « Rx » ne prend la valeur de 1 qu'une seule fois car il n'y a qu'un seul bit à 1 dans le message à transmettre. Puis une fois la réception, nous remarquons bien que le message reçu est égal à 1.

1.2.4 Liaison série avec RTS et CTS

Dans une liaison série asynchrone sur cinq fils (Rx, Tx, GND, RTS, CTS). Les deux signaux (RTS/CTS) supplémentaires servent à assurer le contrôle du flux de transmission. Ils permettent à l'équipement de réception de signaler qu'il est apte ou n'est pas apte à recevoir d'autres données.

Pour cela, l'émetteur doit positionner RTS (Request To Send) à 0, cela indique donc qu'il demande à émettre. Puis le récepteur positionne CTS (Clear To Send) à 0, il indique qu'il est prêt à recevoir. Après ces deux étapes, l'émission peut commencer. L'émetteur signal la fin de l'émission en mettant RTS à 1 et le récepteur acquiesce en repassant CTS à 1. Si l'émetteur refait une requête d'émission, elle n'est pas prise en compte immédiatement car le récepteur est occupé.

2 - Utilisation du protocole CAN

2.1 Réception de l'ID et du DLC d'une trame CAN

Tout d'abord, regardons de quoi se compose une trame CAN afin de pouvoir extraire l'identifiant ainsi que le DLC (Data Length Code) :

Éléments	ID	RTR	DLC	DATA
Nombre de bits	11	1	4	8 à 64

Figure 31 : Éléments d'une trame CAN et son nombre de bits

Ainsi, si nous voulons récupérer l'ID de notre trame, nous devrons récupérer les 8 bits du premier octet et les 3 bits du second octet et pour le DLC les 4 derniers bits du second octet.

```
id = (((unsigned short) can_buffrx[0])<<3 | can_buffrx[1]>>5);
printf("ID : %x\n",id,id);
dlc = can_buffrx[1] & 0x00FF;
printf("DLC : %u\n",(unsigned int) dlc);
```

Figure 32 : Code de réception de l'ID et du DCL d'une trame CAN

Nous récupérons l'ID en prenant les 8 bits du premier octet puis en décalant vers la gauche pour laisser les 3 bits du second octet. Nous récupérons ensuite les 3 bits du premier octet en décalant vers la droite de 5 puis nous faisons un OU logique bit à bit afin d'avoir nos 11 bits d'ID. Concernant le DLC, nous appliquons un ET logique avec le masque 0x00FF soit 00001111 en binaire afin de garder les 4 derniers bits du second octet qui correspondent au DLC. Nous affichons les résultats en hexadécimal.

2.2 Affichage des IDs égaux à 0x310

Pour afficher uniquement les IDs équivalents à 0x310, nous devons modifier :
- ACCEPT_CODE : le code d'acceptation
- ACCEPT_MASK : le masque d'acceptation

Initialement, ACCEPT_CODE = 0x62 en hexadécimal soit **01100010** en binaire et ACCEPT_MASK = 0xFF en hexadécimal soit 11111111 en binaire. Cela permet de filtrer les IDs car si un bit « i » de l'acceptance masque est égal à 0, le bit « i » de l'acceptance code sera décisif pour l'acceptation de la trame. Si un bit « j » de l'acceptance masque est égal à 1, le bit « j » de l'acceptance code n'interviendra pas pour l'acceptation de la trame. De ce fait, si ACCEPT_MASK = 0xFF, alors toutes les trames seront reçues par le nœud.

Si nous souhaitons afficher uniquement 0x310 soit **01100010**000 en binaire, il nous suffit de définir ACCEPT_MASK = 0x62 et ACCEPT_CODE = 0x62 ou 0x00 pour que celui-ci n'intervienne pas.

```
#define ACCEPT_CODE     0x62        /* address (62h -> ? )              */
#define ACCEPT_MASK     0x62        /* mask acceptance of  messages     */
```

Figure 33 : Paramètres pour l'affichage des IDs égaux à 0x310

2.3 Affichages des IDs égaux à 0x320 et 0x330

Pour afficher uniquement les IDs équivalents à 0x320 et à 0x330, nous devons modifier :
- ACCEPT_CODE : le code d'acceptation
- ACCEPT_MASK : le masque d'acceptation

Si nous souhaitons afficher uniquement 0x320 en hexadécimal soit **01100100**000 en binaire et 0x330 en hexadécimal soit **01100110**000, nous modifions il nous suffit de définir ACCEPT_MASK = 0x02 pour que celui-ci « s'en fiche » du deuxième bit et donc que le deuxième bit puisse être égal à 0 ou à 1. ACCEPT_CODE doit quant à lui être égal à **011001X0** avec X = 0 ou 1 en binaire donc ACCEPT_CODE = 0x64 ou 0x66.

```
#define ACCEPT_CODE     0x64        /* address (62h -> ? )              */
#define ACCEPT_MASK     0x02           /* mask acceptance of  messages     */
```

Figure 34 : Paramètres pour l'affichage des IDs égaux à 0x320 et 0x330

2.4 Affichage de deux octets de données sur le barre-graph (ID = 0x370)

Tout d'abord, pour afficher uniquement les IDs égaux à 0x370, nous suivons le même procédé que dans la partie 2.2 Affichage des IDs égaux à 0x310. Nous obtenons alors ACCEPT_MASK = 0x6E et ACCEPT_CODE = 0x6E ou 0x00.

Nous voulons ensuite récupérer les deux premiers octets de données et les stocker dans un short pour les afficher sur notre barre-graph.

```
dataChange = can_buffrx[2]<<8 | can_buffrx[3];
printf("dataChange %u\n",(unsigned int) dataChange);
leds = dataChange;
```

Figure 35 : Code d'affichage de deux octets de données sur la barre-graph

Ainsi les 8 bits du premier octet de donnée se retrouvent côté bits de poids fort (côté gauche) et les 8 bits du second octet de donnée se retrouvent côté bits de poids faible (côté droit).

2.5 Décodage des trames (sauf ID = 0x370)

Notre objectif dans cette partie est de décoder les trames avec les IDs allant de 0x310 à 0x360 en excluant celle valant 0x370. Nous devons donc récupérer les octets de données pour chacune d'entre elles afin de reconstituer un message.

```
if((int) id != 880)
    for(i = 2; i<10; i++)
        printf("%c",can_buffrx[i]);

if((int) id == 864)
    printf("\n");
```

Figure 36 : Code de décodage des trames

Afin d'exclure la trame avec l'ID égal à 0x370, nous faisons un « if » avec la valeur « 880 » car 0x370 en hexadécimal est égal à 880 en décimal. Puis nous récupérons les trames de données pour chaque ID via une boucle for. Enfin, lorsque l'ID est égal à 0x360 ou 864 en décimal, soit la dernière partie du message, nous faisons un saut de ligne pour indiquer la fin. Nous obtenons ainsi le message caché « ATTENTION … ».

2.6 Émission de trames CAN en appuyant sur une touche

Dans cette partie, nous souhaitons envoyer un message lorsque nous appuyons sur une touche. L'ID de notre trame sera de 0x400 soit 10000000000 en binaire. De ce fait le premier octet devra être égal à 10000000 soit 0x80 en hexadécimal que nous retrouvons en effet dans notre code. Ainsi les 3 derniers bits d'ID doivent être égaux à 0 ainsi que le RTR car nous souhaitons transmettre un message. Comme nous souhaitons maintenant transmettre un seul octet de donnée, notre DLC doit être égal à 1 donc 0001 en binaire. Nous obtenons donc comme bits pour le second 000 (ID) 0 (RTR) 0001 (DLC) soit 00000001 donc 0x01 en hexadécimal.

```
if(can_tx_ready)
{
    can_tx_ready = 0;
    toucheClavier = detect_touche();

    if(toucheClavier!=(char)0xff)
    {
        can_tx_ready = 0;
        can_bufftx[0] =  0x80;
        can_bufftx[1]=  0x01;
        can_bufftx[2] = decode_touche(toucheClavier);
        send_message(can_bufftx);
        for(j=0;j<30000;j++){};
    }
}
```

Figure 37 / Code d'émission de messages via l'appui sur une touche

Dans l'octet de donnée, nous avons décidé d'envoyer la valeur de la touche puis nous envoyons notre message via la fonction send_message() et réalisons une boucle for qui fait acte de temporisation bloquante afin de ne pas surcharger le bus CAN.

2.7 Envoie d'un message de taille allant jusqu'à 30 octets

Nous souhaitons maintenant envoyer un message de taille allant jusqu'à 30 octets, pour cela, nous nécessiterons plusieurs trames CAN car une seule trame peut envoyer jusqu'à 8 octets de données. Ainsi pour un message de 30 octets, il faudra alors 4 trames.

```
if(can_tx_ready)
{
    can_tx_ready = 0;
    taille = sizeof(message); /// Taille en octets du message
    can_bufftx[0] =  0x80; /// ID de notre groupe 0x400 en hexadécimal

    for(k = 0; k<taille; k++) /// On parcourt chaque caractère du message
    {
        /// indexBuffer est initialisé à 2
        can_bufftx[indexBuffer] = message[k]; /// On ajoute le caractère dans le prochain octet du buffer Tx
        indexBuffer++; /// On incrémente le buffer pour permettre d'écrire dans le prochain octet

        if(k%8==0||k==taille-1) /// Si on a écrit 8 octets de données, on envoie la 1ère partie du message
        {                       /// Ou si l'on arrive à la fin du message

            if(taille-k<8)      /// Si on est a la fin du message (et inférieur à 8 bits de données)
                (int)can_bufftx[1] =  taille-k; /// On définit la taille d'octets de données à envoyer
            else
                (int)can_bufftx[1] = 0x08; /// Sinon on définit la taille d'octets de données à envoyer à 8

            send_message(can_bufftx); /// On envoie le message
            indexBuffer = 2; /// On définit l'index à 2 pour ré-écrire une nouvelle trame
            for(j=0;j<30000;j++); /// Temporisation bloquante
        }
    }
}
```

Figure 38 : Code d'envoi d'un message de taille allant jusqu'à 30 octets

Pour décrire simplement notre code, nous parcourons chaque caractère du message, tous les 8 octets de données, nous envoyons le message. Pour la dernière partie du message (inférieure ou égal à 8 octets), nous adaptons la taille du DLC s'il y a moins de 8 octets de données à envoyer et nous envoyons la dernière partie via send_message().

3 - TP d'interface PCI-9112

3.1 Acquisition d'un signal issu d'un GBF et affichage des échantillons en temps réel

Tout d'abord, nous devons initialiser notre carte PCI-9112. Pour cela nous appelons la fonction Register_Card() qui renvoie « l'ID » de la carte, si « l'ID » est inférieure à 0, alors l'initialisation de la carte n'a pas fonctionné et dans les autres cas, elle a réussi. Puis nous configurons notre carte, ses entrées et ses sorties. Enfin dans la boucle while, nous lisons le signal transmit par le GBF (sur Vin1). La variable « Value » est un unsigned short codé sur 16 bits. Notre convertisseur analogique numérique, quant à lui, n'est codé que sur 12 bits d'après la datasheet, les 4 autres bits de la variable « Value » sont ainsi réservés pour le multiplexeur. Nous devons donc faire un décalage vers la droite des 4 bits destinés au MUX de « Value ». Puis nous ressortons les échantillons sur le canal N/A (Vout1) grâce au convertisseur numérique - analogique. Ce dernier est relié à un oscilloscope afin de visualiser le signal reconstruit.

```
int main()
{
    ID = Register_Card(PCI_9112, 0);
    if (ID < 0)
        std::cout << "Pas de carte" << std::endl;
    else
        std::cout << "carte trouvee" << std::endl;


    AI_9112_Config(ID, TRIG_INT_PACER);
    AO_9112_Config(ID, 0, AD_B_5_V);

    while (1)
    {
        AI_ReadChannel(ID, 0, AD_B_5_V, &value);
        value = value >> 4;
        AO_WriteChannel(ID, 0, value);
    }

}
```

Figure 39 : Code d'acquisition d'un signal issu d'un GBF et affichage des échantillons

3.2 Écart de tension entre les deux signaux

Lors de la visualisation du signal d'origine produit par le GBF et du signal échantillonné, nous remarquons qu'il y a un écart de tension. Cet écart est dû à la quantification du signal.

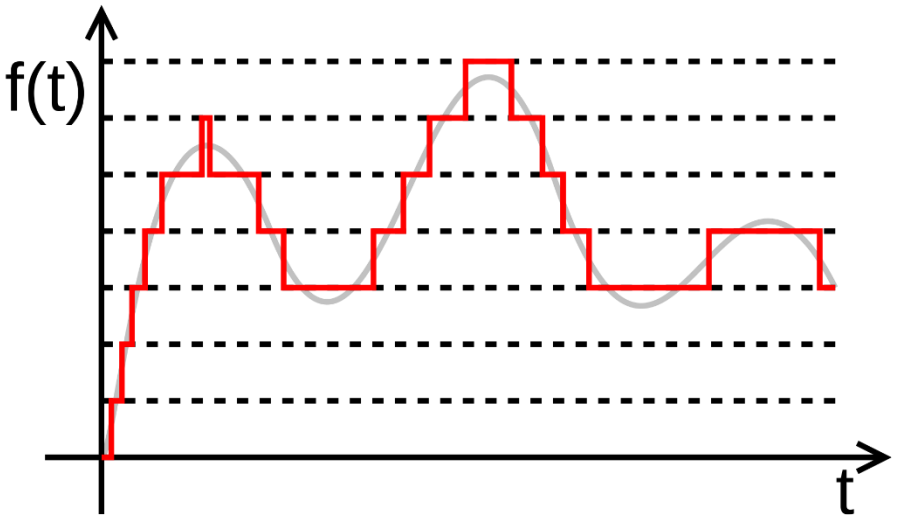


Figure 40 : Exemple de quantification (en rouge) d'un signal continu

En effet, le signal produit par le GBF est quantifié sur 2^{16} niveaux de tension différents, soit 65 536 valeurs possibles alors que le signal échantillonné ne l'est que sur 2^{12}, soit 4 096 niveaux de tension différents. Ainsi, le signal fournit par le GBF possède 2^4 soit 16 fois plus de niveaux de tension que le signal échantillonné ce qui crée un écart de précision et donc un écart de tension entre les deux signaux. Cela est visible sur l'oscilloscope.

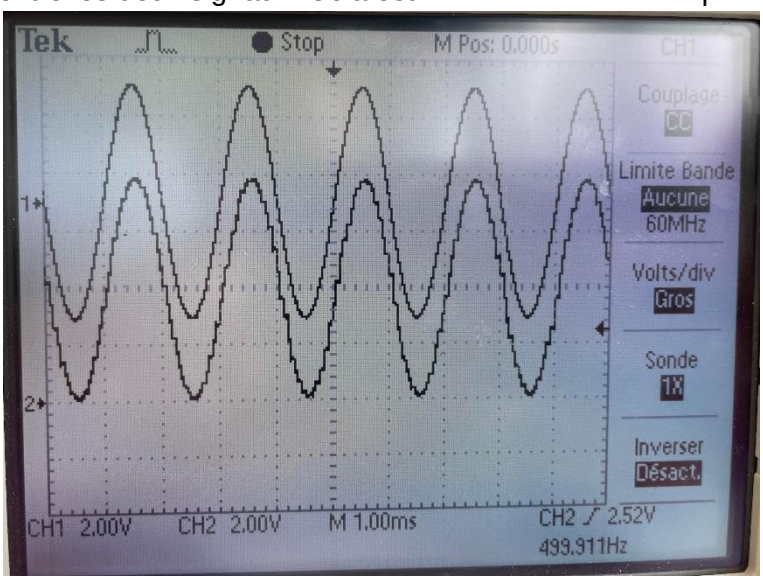


Figure 41 : Écart de tension entre deux signaux

3.3 Explication du nombre maximum d'échantillons par seconde

D'après la datasheet constructeur, la carte PCI-9112 pourrait faire jusqu'à 110 000 échantillons par seconde, mais qu'en est-il réellement ?

D'après le théorème de Shannon qui nous indique que la fréquence d'échantillonnage doit être supérieure au double de la fréquence maximum avec Fe ≥ 2*Fmax. Ainsi, si nous souhaitons faire 110 000 échantillons par seconde, notre Fmax doit être égal à 55 000 Hz soit 55 KHz. Nous paramétrons ainsi notre GBF afin de fournir un signal avec une fréquence d'entrée de 55 KHz cependant notre signal de sortie échantillonné est inexploitable et illisible.

Cela est dû au fait que notre entrée analogique n'a une résolution que de 12 bits soit 4096. Ainsi notre fréquence d'échantillonnage vaut 4 096 (Fe) et donc notre Fmax ne peut pas dépasser 2 048 soit 2 KHz. En effet, lorsque nous demandons au GBF de fournir un signal avec une fréquence d'entrée de 2 KHz, le signal de sortie échantillonné est maintenant exploitable et de bonne qualité. Toute valeur au-dessus entraîne une dégradation du signal.

3.4 Génération logicielle d'un signal de forme triangulaire et affichage des échantillons en temps réel

Afin de générer un signal triangulaire de manière logicielle, nous allons donc supprimer la fonction AI_ReadChannel() qui permettait de récupérer le signal analogique provenant du GBF et allons modifier notre variable « Value ». Sachant que notre convertisseur possède une résolution de 12 bits et que nous voulons un signal triangulaire. Nous souhaitons donc incrémenter « Value » de 0 jusqu'à 4095 (2^{12}-1) afin de faire la pente montante du signal de forme triangulaire. Puis nous devons décrémenter de 4095 à 0 afin de faire la pente descendante du signal de forme triangulaire. Ainsi nous obtenons ce code :

```
value = 1;

while (1)
{
    if (value >= 4094)
        compteur = 1;

    if (value <= 0)
        compteur = 0;

    if (compteur == 0)
        value += 1;
    else
        value -= 1;

    if(value>0&&value<4095)
        AO_WriteChannel(ID, 0, value);
}
```

Figure 42 : Code de génération d'un signal de forme triangulaire

Si nous souhaitons changer la fréquence, nous devons changer le pas d'incrémentation. Plus le pas d'incrémentation est grand, plus la fréquence sera élevée. Par exemple si nous définissons un pas de ±1, nous obtenons une fréquence de 8 Hz d'après l'oscilloscope. Avec un pas de ±30, nous obtenons une fréquence de 4 KHz. Cependant, plus le pas est grand, moins le signal est stable et exploitable.

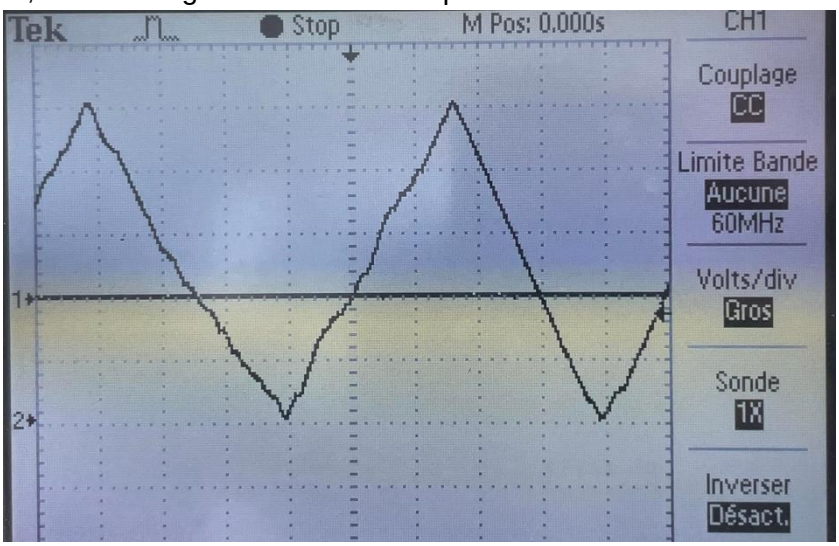


Figure 43 : Génération logiciel d'un signal de forme triangulaire

3.5. Code mesurant la fréquence, l'amplitude et la valeur moyenne d'un signal

Nous souhaitons maintenant créer un programme qui mesure la fréquence, l'amplitude et la valeur moyenne d'un signal toutes les secondes. Pour cela, nous allons utiliser la librairie « chrono » qui permet de mesurer le temps que met un « morceau de code ». Nous avons donc défini une boucle while qui a comme condition de sortie 1 seconde. C'est-à-dire que le code à l'intérieur de la boucle va s'effectuer pendant 1 seconde uniquement puis le programme sort de cette boucle. Nous y prélevons toutes les valeurs de la variable « Value » que nous mettons dans un vector.

```
auto start = high_resolution_clock::now();
auto end = high_resolution_clock::now();
while (duration_cast<seconds> (end - start).count() < 1)
{
    AI_ReadChannel(ID, 0, AD_B_5_V, &value);
    value = value >> 4;
    myBuffer.push_back(value);
    AO_WriteChannel(ID, 0, value);
    end = high_resolution_clock::now();
}
```

Figure 44 : Code qui récupère toutes les valeurs de « Value » chaque seconde

Pour en faire la moyenne, nous additionnons simplement toutes les valeurs de « Value » que nous divisons par le nombre de valeur (taille du vector) :

```
for (size_t i = 0; i < myBuffer.size(); i++)
    sum += myBuffer[i];

moyenne = sum / myBuffer.size();
```

Figure 45 : Code qui calcul la moyenne

Si nous voulons maintenant trouver l'amplitude crête à crête qui est la différence entre l'amplitude maximale et l'amplitude minimale, nous parcourons notre tableau de valeurs de « Value » et nous gardons uniquement la valeur maximale car notre amplitude minimale est de 0 :

```
max = myBuffer[0];

for (size_t i = 0; i < myBuffer.size(); i++)
{
    if (myBuffer[i] > max)
        max = myBuffer[i];
}
```

Figure 46 : Code pour trouver l'amplitude crête à crête

Et enfin, nous voulons trouver la fréquence. Pour cela, nous allons regarder à chaque fois que le signal change de signe, cela indique une demi-période. Dans notre cas, le signal ne changera pas de signe car les valeurs de « Value » vont de 0 à 4095. La valeur médiane est donc de 2048 (valeur équivalent au changement de signe). Nous allons prendre cette valeur comme référence et compter à chaque fois que la prochaine itération dépasse cette valeur (supérieurement comme inférieurement). Puis nous allons diviser ce compteur par deux car lors d'une période, il y a deux changements de signe. Finalement, pour obtenir la période, nous allons diviser par la valeur du compteur. Puis nous savons que la fréquence est équivalente à l'inverse de la période.

```
for (size_t i = 0; i < myBuffer.size(); i++)
{
    if (i != 0)
        if (((myBuffer[i] > 2048) && (myBuffer[i - 1] < 2048)) || ((myBuffer[i] < 2048) && (myBuffer[i - 1] > 2048)))
            count++;
}

count = count / 2;
tpsPeriode = 1 / count;
freq = 1 / tpsPeriode;
```

Figure 47 : Code qui calcul la fréquence

Ainsi, si nous fusionnons chaque morceau de code vu précédemment, nous obtenons notre programme final qui mesure la fréquence, l'amplitude et la valeur moyenne du signal. Voici la totalité du code afin d'obtenir ces trois caractéristiques :

```
auto start = high_resolution_clock::now();
auto end = high_resolution_clock::now();
while (duration_cast<seconds> (end - start).count() < 1)
{
    AI_ReadChannel(ID, 0, AD_B_5_V, &value);
    value = value >> 4;
    myBuffer.push_back(value);
    AO_WriteChannel(ID, 0, value);
    end = high_resolution_clock::now();
}

max = myBuffer[0];
for (size_t i = 0; i < myBuffer.size(); i++)
{
    sum += myBuffer[i];
    if (i != 0)
        if (((myBuffer[i] > 2048) && (myBuffer[i - 1] < 2048)) || ((myBuffer[i] < 2048) && (myBuffer[i - 1] > 2048)))
            count++;

    if (myBuffer[i] > max)
        max = myBuffer[i];
}

count = count / 2;
tpsPeriode = 1 / count;
freq = 1 / tpsPeriode;
moyenne = sum / myBuffer.size();
std::cout << "moy : " << moyenne << " freq : " << freq << " Amp max : " << max << std::endl;

sum = 0;
count = 0;
myBuffer.clear();
```

Figure 48 : Code mesurant la fréquence, l'amplitude et la valeur moyenne d'un signal

Avec ce programme, nous obtenons donc une moyenne de « Value » d'environ 1 600, une fréquence de 500 Hz qui est également celle affichée sur notre oscilloscope (fréquence pour un cas précis) ainsi qu'une amplitude maximale crête à crête d'environ 3168.

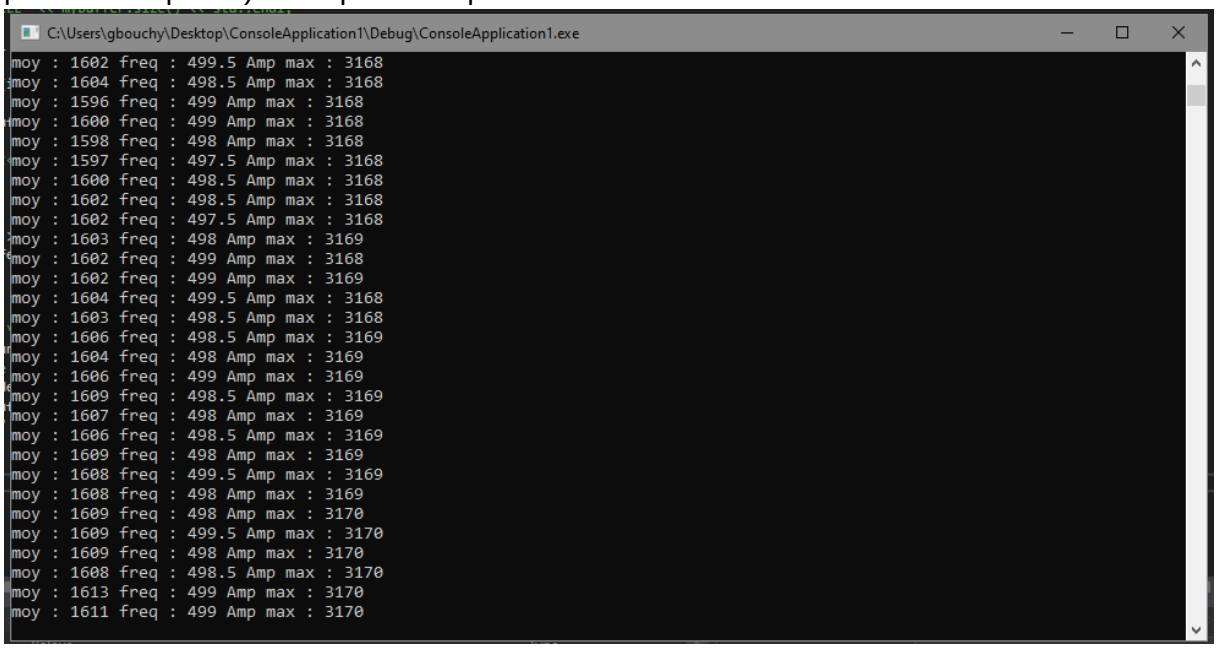


Figure 49 : Capture d'écran de la console affichant la moyenne, la fréquence et l'amplitude

4 - Annexes

4.1 Code pour la mise en œuvre d'une communication série asynchrone

```
/*****************************************************************************
 **      Programme exemple pour le TP æcontroleur                    **
 **                                                                  **
 *****************************************************************************/

#include <reg592.h>                /* registres specifiques au 592 */
#include <stdio.h>
#include "libtp2.h"



/*****************************************/
/* declaration des variables et tableaux */
/*****************************************/
xdata unsigned int LEDS _at_ 0XfD00;
code int TAB[10] = {0x000, 0x001, 0x003,0x00F, 0x010, 0x011, 0x013,0x01F, 0x0FF,
0x0F1};
int i, cpt = 0, flag = 0;
bit sens = 0;

bit FLAG_Timer = 0;
bit transmit = 0;
bit RECEIVED = 0;

int logic_count = 0, logic_countR = 0;
int nb_bit = 0, nb_bitR = 0;
int emet = 0, recept = 0;
char var_tx, char var_rx;

sbit broche_Tx = 0x97; /* broche P1.7 pour l'émission */
sbit broche_Rx = 0x96; /* broche P1.6 pour la réception */
bit Tx,Rx;

/*****************************************/
/* declaration des procedures et fonctions */
/*****************************************/
void tempoBloquante()
{
        for(i =0; i<30000; i++){}
}

////////////////////////////////

int tempoNonBloquante()
{
        cpt++;
        if(cpt==5000)
    {
                cpt = 0;
                return 1;
        }
        else
                return 0;
}

////////////

void chenillard()
{
        if(decode_touche(detect_touche)==9)
                sens = 0;
```

```
        if(decode_touche(detect_touche)==8)
                sens = 1;
}

////////////


void automate_emission(){

        switch(emet)
        {
                case 0:
                        Tx = 1;
                        if(transmit==1)
                                emet = 1;
                        break;
                case 1:
                        Tx=0;
                        nb_bit = 0;
                        logic_count = 0;
                        emet =2;
                        break;
                case 2:
                        if(logic_count==12)
                        {
                                if(nb_bit!=9)
                                        emet = 3;
                                else
                                        emet = 4;
                         }

                        logic_count++;
                        break;
                case 3:
                        logic_count=0;
                        if (nb_bit<8)
            {
                                Tx= var_tx & 0x01;
                                var_tx = var_tx>>1;
                        }
                        else
                                Tx=1;

                        nb_bit++;
                        emet=2;
                        break;
                case 4:
                        transmit=0;
                        emet = 0;
                        break;
        }
}

//////////


void automate_reception()
{
short Var_loc;

        switch(recept)
        {
                case 0:
                        logic_countR =0;
                        if(Rx==0)
                                recept=1;
                        break;
                case 1:
```

```
                        if(logic_countR==5)
                                recept=2;

                        logic_countR++;
                        break;
                case 2:
                        logic_countR = 0;
                        nb_bitR = 0;
                        recept=3;
                        break;
                case 3:
                        if(logic_countR==12)
                        {
                                if(nb_bitR!=8)
                                        recept = 4;
                                else
                                        recept = 5;
                        }

                        logic_countR++;
                        break;
                case 4:
                        nb_bitR++;
                        logic_countR =0;
                        printf("varloc1=%u\n",(unsigned int)Var_loc);
                        Var_loc = Var_loc >> 1;
                        printf("varloc2=%u\n",(unsigned int)Var_loc);

                        if(Rx==1)
            {
                                Var_loc = Var_loc | 0x80;
                                printf("varloc3=%u\n",(unsigned int)Var_loc);
                        }
                        recept = 3;
                        break;
                case 5:
                        if(RECEIVED==0)
            {
                                 var_rx=Var_loc;
                                 printf("message recu=%u\n",(unsigned
int)Var_loc);
                                 RECEIVED=1;
                        }
                        recept = 0;
                        break;
        }
}

////////////
void init_Timer0()
{
        TMOD=TMOD | 2;
        TH0=0; /*tampon pour période maximum ~192µs car quartz µC à 16MHz */
        TL0=0; /* Compteur mis à 0*/
        ET0=1; /* validation de l'interruption Timer0 bit 1 du registre IE */
        EA=1; /* autorisation globale des interruptions bit 7 du registre IE */
        TR0=1;
}

void timer0() interrupt 1
{
         broche_Tx=Tx;
         Rx= broche_Rx;
         automate_emission();
         automate_reception();
}
```

```
//////////////


/*************************************/
/* main program                      */
/*************************************/
char toucheClavier, messageRecu;

void main (void)
{/* debut du programme apres le startup  */
init_Timer0();

while (1)
{
    toucheClavier = detect_touche();
    if(toucheClavier!=(char)0xff)
    {
        var_tx =decode_touche(toucheClavier);
        printf("touche=%u\n",(unsigned int)var_tx);
        transmit = 1;
    }

    if(RECEIVED==1)
    {
        messageRecu = var_rx;
        printf("message recu=%u\n",(unsigned int)messageRecu);
        RECEIVED=0;
    }

    tempoBloquante();

    /*chenillard();
    if(FLAG_Timer==1&&flag==1)
    {
        if(sens==0)
            j++;
        else
            j--;

        leds = tab[j];
        FLAG_Timer = 0;
    }

    if(j==10&&sens==0)
                j=0;

    if(j==0&&sens==1)
                j=10;

    flag = tempoNonBloquante();
    */
  }
}
```

4.2 Code pour l'utilisation du protocole CAN

```
/*-------------------------------------------------------------------------*/
/*                                                                         */
/*      Source file:            code1.C                                    */
/*      Package:                basic                                      */
/*      Software version:       1.4                                        */
/*      Date:                   October 2020                               */
/*      Authors:                A. ELOUARDI & S. BOUAZIZ & Ch. JOUVRAY     */
/*                      based on   Jens-Ulf Pehrs
        */
/*            Philips Semiconductors RHW-PCALH/AI
        */
/*                                                                         */
/*                                                                         */
/*-------------------------------------------------------------------------*/

/*-------------------------------------------------------------------------*/
/*      INCLUDES      D E F I N I T I O N S                                */
/*-------------------------------------------------------------------------*/
#include <reg592.h>
#include <stdio.h>
#include <intrins.h>
#include "candefs.h"
#include "FonctionsBases2.h"
#include "libtp2.h"



/*-------------------------------------------------------------------------*/
/*-------------------------------------------------------------------------*/
/*                                                          FONCTIONS
                                                                */
/*-------------------------------------------------------------------------*/
/*-------------------------------------------------------------------------*/

/*-------------------------------------------------------------------------*/
/*.........................................................................*/
/*               Fonctions pour la configuration et l'initialisation de CAN
        */
/*.........................................................................*/
/*-------------------------------------------------------------------------*/

/*-------------------------------------------------------------------------*/
/*      procedure:      can_config                                         */
/*      input:          ---                                                */
/*      output:         ---                                                */
/*      calls:          ---                                                */
/*-------------------------------------------------------------------------*/

xdata unsigned int LEDS _at_ 0XfD00;

//La reference de tension n'est pas active ... à voir !
unsigned char    can_config ()
{   unsigned char retour=0;
    EA = FALSE;
    can_com=0;
    do    {
        CANADR = CONTROL;
        CANDAT = RESET_REQUEST;
    }   while  ( !(CANDAT & RESET_REQUEST) );
    CANADR = AUTO_ACC_CODE;
    CANDAT = ACCEPT_CODE;
    CANDAT = ACCEPT_MASK;
    CANDAT = BUS_TIM_0;
    CANDAT = BUS_TIM_1;
```

```
    CANDAT = OUT_CONTROL;
    CANCON = DIFFERENTIAL;
    do    {
        CANADR = CONTROL;
        CANDAT = 0;
    }   while  ( CANDAT & RESET_REQUEST );
    CANDAT = CAN_INT_ENABLE;

    if  ( CAN_ERROR_STAT )
                retour=1; //printf("erreur_stat\n");
    PS1 = TRUE;
    ES1 = TRUE;
    EA  = TRUE;
        return retour;
}

/*************************************************************************/
/*      procedure:      can_contr_init                                   */
/*      input:          ---                                              */
/*      output:         ---                                              */
/*      calls:          led_init, can_config, slio_init                  */
/*************************************************************************/

void can_contr_init()
{
    can_config ();
}


/*************************************************************************/
/*      procedure:      CANInitialisation                                */
/*      input:          ---                                              */
/*      output:         ---                                              */
/*      calls:          can_contr_init
*/
/*************************************************************************/

void CANInitialisation()
{
        can_contr_init();
    can_init_request=FALSE;
        int_save = CANCON;
        can_tx_ready=1;
}




/*************************************************************************/
/*.......................................................................*/
/*                      Fonction en cas de problème sur le CAN
                                */
/*.......................................................................*/
/*************************************************************************/



/*************************************************************************/
/*      procedure:      can_int_error                                    */
/*      input:          ---                                              */
/*      output:         ---                                              */
/*      calls:          ---                                              */
/*-----------------------------------------------------------------------*/
/*      running on interrupt level, called from "can_int ()"             */
/*************************************************************************/
```

```
void    can_int_error ()
{
    if  ( CAN_ERROR_STAT )  {
        can_event       = TRUE;
        if  ( CAN_BUS_STATUS )
            can_system_error = TRUE;
    }
    else  {
        can_event    = FALSE;
    }
}

/*************************************************************************/
/*.......................................................................*/
/*                        Fonctions pour la réception de messages sur le CAN
                */
/*.......................................................................*/
/*************************************************************************/

/*************************************************************************/
/*      procedure:      can_buf_full                                     */
/*      input:          ---                                              */
/*      output:         ---                                              */
/*      calls:          ---                                              */
/*-----------------------------------------------------------------------*/
/*      running on interrupt level, called from "can_int ()"             */
/*-----------------------------------------------------------------------*/
/*                                                                       */
/*                                                                       */
/*************************************************************************/

void    can_buf_full ()
{

//    CANSTA = (char *)tab_rx;
    CANSTA = (unsigned char)(can_buffrx);
         CANADR = DMA_EN|AUTO_RX_BUFFER;
         _nop_();
         _nop_();
         _nop_();
         _nop_();
         _nop_();
    if  ( overrun_int )
        CANCON = RELEASE_RX_BUF | OVERRUN_CLEAR;
    else
         CANCON = RELEASE_RX_BUF;

          can_event    = TRUE;
    can_rx_ready =TRUE;
}

/*************************************************************************/
/*.......................................................................*/
/*                        Fonctions pour l'émission de messages sur le CAN
                */
/*.......................................................................*/
/*************************************************************************/

/*************************************************************************/
/*      procedure:      send_message                                     */
/*      input:          data_table                                           */
/*      output:         ---                                              */
/*      calls:                                                           */
/*************************************************************************/

unsigned char send_message ( unsigned char *tabtx ){
```

```
  int_save    = CANCON;
  if  ( CAN_TX_BUFFER )   {
    CANSTA = (char)tabtx;
    CANADR = DMA_EN|AUTO_TX_BUFFER;
         _nop_();
         _nop_();
         _nop_();
         _nop_();
         _nop_();
    can_event = TRUE;
    can_tx_ready = FALSE;
    can_tx_sending = TRUE;
    CANCON = TRANSMIT_REQUEST|DIFFERENTIAL;                    /* start CAN
transmit to SLIO */
         return 1;
  }
  else   {                                                     /* error exit,
if transmit    */

  //ATTENTION ne pas faire ca car cela risque de planter
  // il faut rien faire dans ce cas et prévenir qu'il n'y  pas
  //d'émission..... et emettre à nouveau apres.
    CANCON          = TRANSMIT_ABORT|DIFFERENTIAL;     /*    not possible
*/
    can_system_error = TRUE;
    can_init_request = TRUE;
    can_partner      = FALSE;
    can_tx_ready=can_tx_sending=FALSE;
         return 0;
    };
}

/*************************************************************************/
/*      procedure:      can_tx_complete                                  */
/*      input:          ---                                              */
/*      output:         ---                                              */
/*      calls:          ---                                              */
/*-----------------------------------------------------------------------*/
/*      running on interrupt level, called from "can_int ()"             */
/*************************************************************************/

void    can_tx_complete ()
{
    can_event = TRUE;
         can_tx_ready = TRUE;
}


/*************************************************************************/
/*.......................................................................*/
/*                        Fonction gérant les interruptions sur le CAN
                          */
/*.......................................................................*/
/*************************************************************************/
/*      procedure:      can_int                                          */
/*      input:          ---                                              */
/*      output:         ---                                              */
/*      calls:          ---                                              */
/*-----------------------------------------------------------------------*/
/*      running on interrupt level, called from "can_int ()"             */
/*************************************************************************/

void    can_int (void)  interrupt 5
{
    int_save = CANCON;

    if  ( error_int ){                    // due à une erreur
```

```
                can_int_error ();
        }
    if  ( receive_int )
        // due à la réception d'un message
        {
                // on stocke la valeur du tps lors de la réception du message NON
                    CANSTA = (unsigned char)(can_buffrx);
                        CANADR = DMA_EN|AUTO_RX_BUFFER;
                        _nop_();
                        _nop_();
                        _nop_();
                        _nop_();
                if  ( overrun_int )
                CANCON = RELEASE_RX_BUF | OVERRUN_CLEAR;
                else
                CANCON = RELEASE_RX_BUF;
                        can_event    = TRUE;
                can_rx_ready =TRUE;

        }
    if  ( transmit_int && CAN_TX_COMPLETE ){// due à une transm- correcte et buffer
d'emission vide
                can_tx_complete ();
        }
}


/*************************************************************************/
/*      procedure:      main                                             */
/*      input:          ---                                              */
/*      output:         ---                                              */
/*      calls:          CANInitialisation, LireBufferReception           */
/*                      CANEmissionMessage, CANREceptionMessage          */
/*************************************************************************/

unsigned int countx;
int taille;
        char message[32] = {'s','a','l','u','t'};
unsigned char can_bufftx[20];
int count=0;
char toucheClavier,touche;
/**************************   MAIN      *********************************/
void main()
{
        short id;
        short dataChange;
        char data1;
        char data2;
        char dlc;
        int i,j,k;



        int indexBuffer=2;
        //Initialisation du CAN
    CANInitialisation();
        printf("initialisation du systeme\n");

        while(1)
        {
                /**** erreur donc demande d'initialisation *****/
                if  ( can_system_error ){
                        printf("CAN ERREUR\n");
                        can_init_request = TRUE;
                }
                /**** besoin d'une initialisation *****/
                if  ( can_init_request ){
```

```
                        printf("initialisation CAN\n");
                    CANInitialisation();
                        can_init_request=FALSE;
        }

                if (can_rx_ready)
                {
                  can_rx_ready=0;

            /// Réception de l'ID et du DLC d'une trame CAN ///
                        /*id = (((unsigned short) can_buffrx[0]) <<3 |
can_buffrx[1]>>5);
                          printf("ID : %x\n",id);
                          dlc = can_buffrx[1] & 0x00FF;
                          printf("DLC : %u\n",(unsigned int) dlc);*/

            /// Affichage de deux octets de données sur le barre-graph ///
            /*dataChange = can_buffrx[2]<<8 | can_buffrx[3];
                        printf("dataChange %u\n",(unsigned int) dataChange);
                        leds = dataChange;*/

                        /// Décodage des trames (sauf ID = 0x370) ==> Message
secret ///
                        if((int) id != 880)
                        {
                                for(i = 2; i<10; i++)
                                {
                                        printf("%c",can_buffrx[i]);
                                }
                        }
                        if((int) id == 864)
                        {
                                printf("\n");
                        }
                }


                if(can_tx_ready)
        {
            /// Émission de trames CAN en appuyant sur une touche ///
            /*toucheClavier = detect_touche();

            if(toucheClavier!=(char)0xff)
            {
                can_tx_ready = 0;
                can_bufftx[0] =  0x80;
                can_bufftx[1]=  0x01;
                can_bufftx[2] = decode_touche(toucheClavier);
                send_message(can_bufftx);
                for(j=0;j<30000;j++) {};
            }
            */

            /// Envoie d'un message de taille allant jusqu'à 30 octets  ///
            can_tx_ready = 0;
            taille = sizeof(message); /// Taille en octets du message
            can_bufftx[0] = 0x80; /// ID de notre groupe 0x400 en hexadécimal

            for(k = 0; k<taille; k++) /// On parcourt chaque caractère du message
            {
                /// indexBuffer est initialisé à 2
                can_bufftx[indexBuffer] = message[k]; /// On ajoute le caractère
dans le prochain octet du buffer Tx
                indexBuffer++; /// On incrémente le buffer pour permettre d'écrire
dans le prochain octet
```

```
                if(k%8==0||k==taille-1) /// Si on a écrit 8 octets de données, on
envoie la 1ère partie du message
                {                       /// Ou si l'on arrive à la fin du message

                    if(taille-k<8)      /// Si on est a la fin du message (et
inférieur à 8 bits de données)
                        (int)can_bufftx[1] =  taille-k; /// On définit la taille
d'octets de données à envoyer
                    else
                        (int)can_bufftx[1] = 0x08; /// Sinon on définit la taille
d'octets de données à envoyer à 8

                    send_message(can_bufftx); /// On envoie le message
                    indexBuffer = 2; /// On définit l'index à 2 pour ré-écrire une
nouvelle trame
                    for(j=0;j<30000;j++); /// Temporisation bloquante
                }
            }
                }
          }
}
```

4.3 Code d'interface PCI-9112

```
// ConsoleApplication1.cpp : Ce fichier contient la fonction 'main'. L'exécution du programme commence et se termine à cet endroit.

#include <iostream>
#include <windows.h>
#include "Dask.h"
#include <vector>
#include <chrono>
#include <bits.h>

using namespace std;
using namespace chrono;

short ID;
unsigned short value;
int x = 0;
int compteur = 0;
std::vector<int> myBuffer;

int main()
{
	ID = Register_Card(PCI_9112, 0);
	if (ID < 0)
		std::cout << "Pas de carte" << std::endl;
	else
		std::cout << "carte trouvee" << std::endl;

	AI_9112_Config(ID, TRIG_INT_PACER);
	AO_9112_Config(ID, 0, AD_B_5_V);

	value = 1;

	long sum = 0;
	long moyenne;
	double count = 0;
	double tpsPeriode, freq;
	short max;

	while (1)
	{
		//Q1
		/*
		AI_ReadChannel(ID, 0, AD_B_5_V, &value);
		value = value >> 4;
		AO_WriteChannel(ID, 0, value);
		*/

		//Q2
		/*
		if (value >= 4094)
			compteur = 1;

		if (value <= 0)
			compteur = 0;

		if (compteur == 0)
			value+=30;
		else
			value-=30;

		if(value>0&&value < 4095)
			AO_WriteChannel(ID, 0, value);
		*/

		//Q3
```

```
                auto start = high_resolution_clock::now();
                auto end = high_resolution_clock::now();
                while (duration_cast<seconds> (end - start).count() < 1)
                {
                        AI_ReadChannel(ID, 0, AD_B_5_V, &value);
                        value = value >> 4;
                        myBuffer.push_back(value);
                        AO_WriteChannel(ID, 0, value);
                        end = high_resolution_clock::now();
                }

                max = myBuffer[0];
                for (size_t i = 0; i < myBuffer.size(); i++)
                {
                        sum += myBuffer[i];
                        if (i != 0)
                                if (((myBuffer[i] > 2048) && (myBuffer[i - 1] <
2048)) || ((myBuffer[i] < 2048) && (myBuffer[i - 1] > 2048)))
                                        count++;

                        if (myBuffer[i] > max)
                                max = myBuffer[i];
                }

                count = count / 2;
                tpsPeriode = 1 / count;
                freq = 1 / tpsPeriode;
                moyenne = sum / myBuffer.size();
                std::cout << "moy : " << moyenne << " freq : " << freq << " Amp
max : " << max << std::endl;

                sum = 0;
                count = 0;
                myBuffer.clear();
        }

        //Release_Card(ID);
        return 0;

}

// Exécuter le programme : Ctrl+F5 ou menu Déboguer > Exécuter sans débogage
// Déboguer le programme : F5 ou menu Déboguer > Démarrer le débogage

// Astuces pour bien démarrer :
//   1. Utilisez la fenêtre Explorateur de solutions pour ajouter des fichiers et
les gérer.
//   2. Utilisez la fenêtre Team Explorer pour vous connecter au contrôle de code
source.
//   3. Utilisez la fenêtre Sortie pour voir la sortie de la génération et d'autres
messages.
//   4. Utilisez la fenêtre Liste d'erreurs pour voir les erreurs.
//   5. Accédez à Projet > Ajouter un nouvel élément pour créer des fichiers de
code, ou à Projet > Ajouter un élément existant pour ajouter des fichiers de code
existants au projet.
//   6. Pour rouvrir ce projet plus tard, accédez à Fichier > Ouvrir > Projet et
sélectionnez le fichier .sln.
```